

Smart contract audit

rain.verify.interface

Content

Project description	3
Executive summary	3
Reviews	3
Security findings	5
**** Critical	5
*** High	5
** Medium	5
* Low	5
Informational	5
Approach and methodology	6



Project description

The repository `rain.verify.interface` contains minimal contract implementation that always returns “approved” for every account and timestamp, used as a no-op verifier.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.verify.interface

Reviews

Date	Repository	Commit
10/02/26	<code>rain.verify.interface</code>	<code>f5f3bf39d6ab70cd177692a39b20eae6143899bf</code>

Scope

Contracts
<code>src/concrete/VerifyAlwaysApproved.sol</code>
<code>src/interface/IVerifyCallbackV1.sol</code>
<code>src/interface/IVerifyV1.sol</code>



Technical analysis and findings

Critical	0
High	0
Medium	0
Low	0
Informational	0



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No medium severity issue found.

* Low

No low severity issue found.

Informational

No informational severity issue found.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.